

Santas Workshop

Every year, Kaggle (kaggle.com) posts an optimization competition, always about helping Santa. In 2019, it was about planning workshop visits to Santa's workshop. The problem is a so-called Generalized Assignment problem.

Problem

- Minimize Santa Claus's visit costs (more money for presents !)

Sets

- Families: $f = 1, 2, \dots, F$ with $F = 5000$ or $F = 1000$.
- Days: $d = 1, 2, \dots, D$ with $D = 100$ or $D = 20$

Parameters

- $FamilySize_f$: The number of members of family f
- $DayVisitCost_{f,d}$: Cost of assigning family f to day d

Decision variables

- $x_{f,d} \in \{0, 1\}$: 1 if family f visits the workshop on day d and 0 otherwise.

Model

Objective:

- Minimize total costs for the visits to Santa's workshop:

$$\sum_{f,d} DayVisitCost_{f,d} \cdot x_{f,d}$$

Constraints:

- Each family **HAS** to get a day:

$$\sum_d x_{f,d} = 1 \quad \forall f$$

- Each day, there has to be at least 125 persons visiting the workshop:

$$\sum_f FamilySize_f \cdot x_{f,d} \geq 125 \quad \forall d$$

- There can at most be 300 visitors pr day:

$$\sum_f FamilySize_f \cdot x_{f,d} \leq 300 \quad \forall d$$

The full model in Julia/JuMP, available with the name

`SantasWorkshop.jl`

from the book web-site, is given below:

```
*****
# Santas Workshop Assignment, Cannery example,
# "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
#using Gurobi
*****

*****
# Read data
# Data:
# F : No Families
# D : No Days
# FamilySize[f] : No of persons in a family
# DayVisitCost[f,d0] : Cost for the family visiting Santa that day
include("SantasWorkshopData_5000_100.jl")
#include("SantasWorkshopData_1000_20.jl")
println("No Families: ", F)
println("No Days: ", D)
*****
```

```

*****
# Model
santa = Model(HiGHS.Optimizer)
#santa = Model(Gurobi.Optimizer)

@variable(santa, x[f=1:F,d=1:D],Bin)

# minimize costs of visits
@objective(santa, Min,
           sum( DayVisitCost[f,d]*x[f,d] for f=1:F,d=1:D) )

# all families get to visit once
@constraint(santa, [f=1:F],
           sum( x[f,d] for d=1:D) == 1)

# Day lower bound
@constraint(santa, [d=1:D],
           sum( FamilySize[f]*x[f,d] for f=1:F) >= 125)

# Day upper bound
@constraint(santa, [d=1:D],
           sum( FamilySize[f]*x[f,d] for f=1:F) <= 300)

#print(santa)
*****

*****
# solve
#set_silent(santa)
#set_optimizer_attributes(santa, "MIPGap" => 0.0)
println("Solving Model")
optimize!(santa)
*****

*****
# Report results
println("-----");
if termination_status(santa) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(santa))\n")
else
    println(" No solution")
end
println("-----");

```

Comments to the Julia/JuMP program:

- Notice: The data-set files are included into the program using the "include()" statement.
- Notice: We comment out one of the two include statement, this makes it easy to switch between the data-sets, by simply changing which line is commented out.
- Notice: In the same way, the model statement which assigns the solver, has a commented out line which uses Gurobi instead of HiGHS. Gurobi offers free licenses for students, but installing Gurobi and acquiring a personal license takes some time. You do not need Gurobi for this course, but if you have it already or plan to take other OR courses you can try it out and solve the big instance with all families and days.